



Gymnázium J. K. Tyla

Úvod do agentického AI a harness pro automatizovaný softwarový vývoj

ODBORNÁ PRÁCE

Autor práce: Patrik Marius, 4.D

Vedoucí práce: Michal Dočekal

Prohlášení

Prohlašuji, že jsem tuto studentskou odbornou práci vypracoval/a samostatně pod dohledem vedoucího uvedeného na první straně. Všechny použité zdroje jsou uvedeny v seznamu zdrojů a informace z nich získané jsou v textu řádně označeny odkazem na zdroj. Souhlasím s tím, aby tištěná forma práce byla uchována na Gymnázium J. K. Tyla a tam používána jako tištěný zdroj např. pro další studentské práce či pro prezentaci vzdělávání na GJKT.

V Hradci Králové dne

Podpis autora práce:

.....

Anotace

Tato odborná práce se zabývá principy agentního inženýrství (*agentic engineering*) a architekturou řídicích harnessů pro automatizovaný vývoj softwaru. Praktickým přínosem práce je návrh a implementace systému DarkFactory — agentního harnessu instalovatelného jako aplikace pro platformu GitHub (GitHub App). Systém usiluje o maximální možnou míru automatizace vývojového cyklu od sémantické analýzy požadavků v GitHub Issues, přes technické plánování, až po generování kódu a vystavení pull requestu. Práce reflektuje, že současné agentní systémy nelze vnímat jako plně autonomní: jazykové modely vyžadují deterministické mantinely proti uvíznutí v nekonečných cyklech, správu kontextu bez sémantického posunu a především kontinuální zapojení člověka formou schvalovacích bran (*Human-in-the-loop*).

Klíčová slova

agentní inženýrství, řídicí harness, GitHub App, DarkFactory, automatizace vývoje, orchestrace agentů, jazykové modely

Annotation

This thesis explores the foundational principles of agentic engineering and harness architecture for automated software development. The practical contribution of the work is the design and implementation of DarkFactory — an agentic harness installable as a GitHub App. The system aims to maximize the automation of the software engineering lifecycle, ranging from issue interpretation in GitHub Issues and technical planning to code generation and pull request delivery. The thesis emphasizes that modern agentic systems cannot be considered fully autonomous: large language models require deterministic guardrails against infinite loops, rigorous context management without semantic drift, and indispensable human-in-the-loop governance via approval gates.

Keywords

agentic engineering, harness architecture, GitHub App, DarkFactory, software development automation, agent orchestration, language models

Obsah

Prohlášení	2
Anotace	3
Klíčová slova	3
Annotation	3
Keywords	3
1 Úvod	7
1.1 Motivace a vymezení problému	7
1.2 Cíl práce a výzkumné otázky	7
1.3 Metodika práce	7
2 Teoretická část	8
2.1 Git a GitHub (Správa verzí)	8
2.1.1 Úvod	8
2.1.2 Větvě (Branches) a izolace kódu	8
2.1.3 Model pull requestu (PR)	8
2.1.4 Slučování změn (Squash and Merge)	8
2.1.5 Kontinuální integrace (CI a GitHub Actions)	8
2.1.6 Požadované kontroly (Required Checks) a ochrana větví	8
2.2 LLM, chatboti a agenti	8
2.2.1 Úvod	8
2.2.2 Tokeny, tokenizace a embedding	9
2.2.3 Tahy a správa KV cache	10
2.2.4 Kompakce kontextu a ztrátová komprese	10
2.2.5 Sémantický posun (Semantic Drift)	10
2.2.6 Alternativní paměťové architektury (RAG a stavový graf)	10
2.2.7 Degradace pozornosti (Context Rot)	10
2.2.8 Promptové inženýrství a negativní instrukce	10
2.3 Harness a agentní inženýrství (prompt, kontext, smyčka, graf...)	10
2.3.1 Úvod	10
2.3.2 Agent vs. Chatbot	10
2.3.3 Agentní smyčka a prováděcí cyklus ReAct	11
2.3.4 Běhové prostředí nástrojů a pískoviště (Sandbox)	11
2.3.5 Patologie divergence: perseverace a oscilace	11
2.3.6 Dovednosti (Skills)	11
2.3.7 Model Context Protocol (MCP servery)	11

2.3.8 Škálování: hierarchičtí subagenti a DAG workflow	11
2.3.9 Zapojení člověka do smyčky (Human-in-the-loop)	11
3 Praktická část	12
3.1 Cíl a rozsah systému DarkFactory	12
3.2 Architektura a životní cyklus požadavku	12
3.3 Orchestrační jádro a validační infrastruktura	12
3.4 Systém revizních značek pro lidský dohled nad akademickým textem	12
4 Výsledky a diskuse	13
4.1 Sjednocení pracovních postupů a přenositelnost napříč doménami	13
4.2 Provozní metriky a spolehlivost agentních běhů	13
4.3 Poznatky a systémová úskalí z provozu	13
4.4 Diskuse: Porovnání architektur	13
4.5 Systémová a metodická omezení	13
5 Závěr	14
Seznam příloh	15
A Obsah přiloženého média	16
B Schéma konfiguračního manifestu darkfactory.json	17
C Sdílené workflow pro GitHub Actions	18
D Systémové prompty plánovacího a kódovacího agenta	19
E Protokol revizních značek v sazebním systému Typst	20
Seznam zdrojů	21

Seznam obrázků a tabulek

Obrázek 1	Architektura autonomní ReAct smyčky (Reasoning + Acting) a tok dat mezi uživatelé, kontextem, modelem a výkonným prostředím.	11
Tabulka 1	Rozpoznávání prostředí podle souboru popisujícího balíček.	12

1 Úvod

1.1 Motivace a vymezení problému

1.2 Cíl práce a výzkumné otázky

1.3 Metodika práce

2 Teoretická část

2.1 Git a GitHub (Správa verzí)

2.1.1 Úvod

Pro autonomní vývoj softwaru je spolehlivá správa verzí naprosto nezbytným základem. Jazykové modely generují kód na základě statistické pravděpodobnosti, a proto se nevyhnutelně dopouštějí chyb, logických přehmatů či regresí. Verzovací systém vytváří bezpečné a deterministické prostředí, v němž lze každou úpravu zaznamenat, otestovat a v případě selhání kdykoliv vrátit zpět k funkčnímu stavu. Namísto teoretických abstrakcí práce přímo využívá distribuovaný systém **Git** v kombinaci s platformou **GitHub**.

Klíčové komponenty infrastruktury zahrnují:

- **Distribuovaný systém Git (1):** Ukládá kompletní historii projektu v podobě jednotlivých revizí (*commitů*). Vývojář i agent pracují s plnou lokální kopií repozitáře, což umožňuje provádět změny, přepínat větve a spouštět lokální testy zcela nezávisle na síťovém připojení.
- **Platforma GitHub:** Slouží jako centrální bod pro sdílení kódu, týmovou koordinaci a automatizaci:
 - **Zadávání a sledování úkolů (Issues):** Strukturovaná textová zadání požadavků a hlášení chyb, která agentovi slouží jako výchozí specifikace úlohy.
 - **Revize změn (Pull Requests):** Uživatelské rozhraní pro přehledné zobrazení diffu, diskusi nad kódem a formální schválení člověkem.
 - **Automatizace (GitHub Actions):** Běhové prostředí pro automatické spouštění testů, linterů a překladů při každé události v repozitáři.

Agent v tomto pojetí nevystupuje jako černá skříňka s proprietárním protokolem, nýbrž jako standardní přispěvatel, který plně respektuje běžné vývojářské zvyklosti a nástroje.

2.1.2 Větve (Branches) a izolace kódu

2.1.3 Model pull requestu (PR)

2.1.4 Slučování změn (Squash and Merge)

2.1.5 Kontinuální integrace (CI a GitHub Actions)

2.1.6 Požadované kontroly (Required Checks) a ochrana větví

2.2 LLM, chatboti a agenti

2.2.1 Úvod

V agentním softwarovém inženýrství vystupuje velký jazykový model (LLM) jako stochastické kognitivní jádro celého systému. Z hlediska vnitřní architektury se jedná o dekodérový

transformer (*Decoder-only*), jehož typickými představiteli jsou moderní modely řad Claude, GPT či DeepSeek (2). Role modelu nespočívá ve vystupování jako vševědoucí orákulum se spolehlivou znalostí okolního světa, nýbrž jako pokročilý generátor hypotéz, kódu a strukturovaných volání nástrojů řízený obdrženým kontextem.

Základní principy fungování modelu zahrnují:

- **Autoregresivní predikce:** Model zpracovává zadanou sekvenci textu a na jejím základě iterativně předpovídá nejpravděpodobnější následující symboly (tokeny).
- **Stochastická povaha:** Vzhledem k pravděpodobnostnímu vzorkování může model na totožný vstup reagovat mírně odlišně, což vyžaduje deterministické mantinely v nadřazeném řídicím harnessu.

Pro efektivní nasazení modelu do vývojového cyklu je nezbytné porozumět způsobu, jakým reprezentuje informace a jaké fyzické limity vymezují jeho operační paměť.

2.2.2 Tokeny, tokenizace a embedding

Jazykový model nepracuje přímo se znaky ani slovy v lidském slova smyslu. Vstupní text je nejprve deterministickým algoritmem převeden na číselné reprezentace, se kterými následně počítají maticové vrstvy neuronové sítě.

Tento proces zahrnuje následující pojmy:

- **Tokeny a tokenizér:** Token představuje základní diskrétní jednotku (celé slovo, slabiku či fragment znaků). Převod mezi textem a posloupností číselných tokenů zajišťuje tokenizér (nejčastěji na bázi algoritmu Byte-Pair Encoding, BPE).
- **Embedding:** Každý token je promítnut do vícerozměrného vektorového prostoru, kde geometrická vzdálenost a úhel vektorů vyjadřují sémantickou příbuznost pojmů (např. vektorová analogie král – muž + žena $\approx$ královna).
- **Jazyková asymetrie tokenizace:** Vzhledem k trénovacím datům optimalizovaným primárně pro angličtinu spotřebovávají flektivní jazyky s bohatou diakritikou (včetně češtiny) 2× až 3× více tokenů pro vyjádření téhož významu.

Z inženýrského hlediska je proto žádoucí vést systémové prompty, technické plány i komunikaci mezi nástroji v angličtině, aby se šetřila kapacita kontextu a snížila latence inference.

2.2.3 Tahy a správa KV cache

2.2.4 Kompakce kontextu a ztrátová komprese

2.2.5 Sémantický posun (Semantic Drift)

2.2.6 Alternativní paměťové architektury (RAG a stavový graf)

2.2.7 Degradace pozornosti (Context Rot)

2.2.8 Promptové inženýrství a negativní instrukce

2.3 Harness a agentní inženýrství (prompt, kontext, smyčka, graf...)

2.3.1 Úvod

V terminologii agentního inženýrství označuje pojem **harness** (řídící postroj) aplikační vrstvu, která obklopuje samotné inferenční jádro jazykového modelu. Samotné inferenční jádro provádí výhradně matematické maticové operace nad zadanými váhami a vektory tokenů; veškerou orchestraci, práci se soubory a řízení bezpečnosti zajišťuje harness. Ústřední komponentou a hlavní prováděcí funkcí, která v architektuře harnessu řídí samotný běh a iterativní koordinaci agenta v reálném vývojovém prostředí, je takzvaná **agentní smyčka** (*Agent Loop*).

2.3.2 Agent vs. Chatbot

Rozdíl mezi konverzačním chatbotem a autonomním agentem nespočívá v odlišném jazykovém modelu, ale v architektuře jeho zapojení do pracovního prostředí. Zatímco chatbot funguje pasivně jako textový rádce, agent vystupuje jako aktivní vykonavatel úkolů. Srovnání obou přístupů:

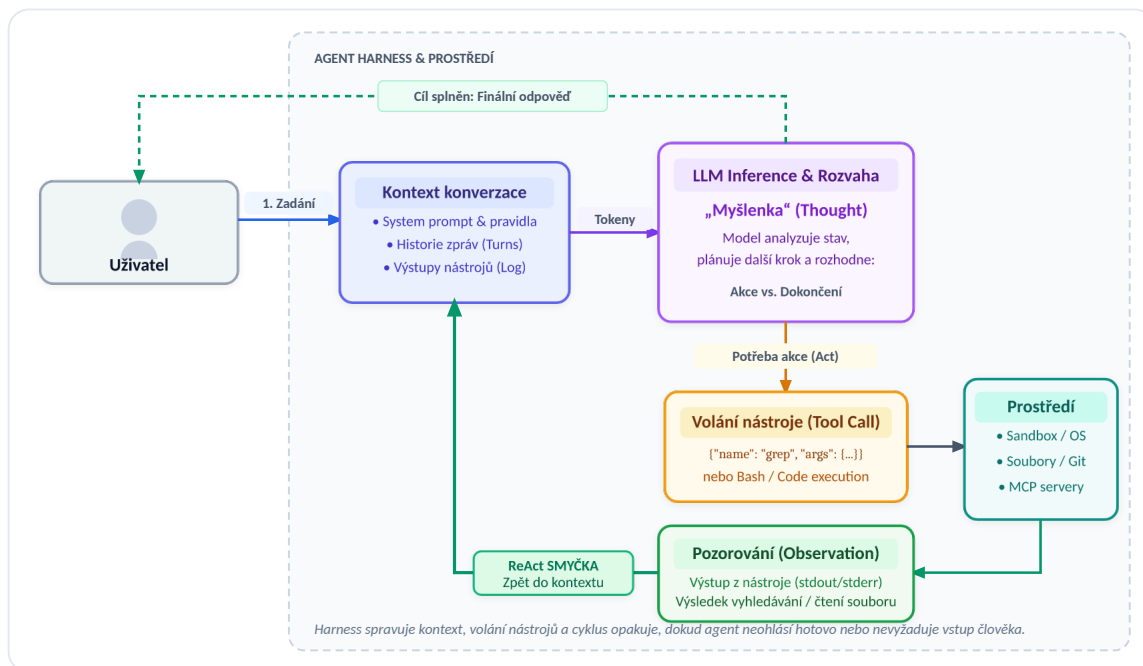
- **Konverzační chatbot:**

- Reaguje pouze na přímé textové výzvy v uzavřeném okně chatu.
- Nemá přímý přístup k souborovému systému ani k nástrojům operačního systému.
- Uživatel musí navržený kód ručně zkopírovat, vložit do projektu a otestovat.

- **Autonomní agent:**

- Je vybaven sadou výkonných nástrojů (*tools*) pro práci s repozitářem.
- Aktivně prozkoumává soubory, modifikuje zdrojový kód, spouští testy a interpretuje jejich návratové kódy.
- Funguje v autonomní prováděcí smyčce, v níž iterativně reaguje na reálnou odezvu vývojového prostředí.

2.3.3 Agentní smyčka a prováděcí cyklus ReAct



Obrázek 1: Architektura autonomní ReAct smyčky (Reasoning + Acting) a tok dat mezi uživatelem, kontextem, modelem a výkonným prostředím.

2.3.4 Běhové prostředí nástrojů a pískoviště (Sandbox)

2.3.5 Patologie divergence: perseverace a oscilace

2.3.6 Dovednosti (Skills)

2.3.7 Model Context Protocol (MCP servery)

2.3.8 Škálování: hierarchičtí subagenti a DAG workflow

2.3.9 Zapojení člověka do smyčky (Human-in-the-loop)

3 Praktická část

3.1 Cíl a rozsah systému DarkFactory

3.2 Architektura a životní cyklus požadavku

Soubor	Prostředí	Doména
<code>pyproject.toml</code>	Python	kód
<code>Cargo.toml</code>	Rust	kód
<code>package.json</code>	Node	kód
<code>go.mod</code>	Go	kód
<code>typst.toml</code>	Typst	text
<code>.latexmkrc</code>	LaTeX	text
<code>lakefile.toml</code>	Lean	matematika

Tabulka 1: Rozpoznávání prostředí podle souboru popisujícího balíček.

3.3 Orchestrační jádro a validační infrastruktura

3.4 Systém revizních značek pro lidský dohled nad akademickým textem

4 Výsledky a diskuse

4.1 Sjednocení pracovních postupů a přenositelnost napříč doménami

4.2 Provozní metriky a spolehlivost agentních běhů

4.3 Poznatky a systémová úskalí z provozu

4.4 Diskuse: Porovnání architektur

4.5 Systémová a metodická omezení

5 Závěr

Seznam příloh

A	Obsah přiloženého média	16
B	Schéma konfiguračního manifestu darkfactory.json	17
C	Sdílené workflow pro GitHub Actions	18
D	Systémové prompty plánovacího a kódovacího agenta	19
E	Protokol revizních značek v sazebním systému Typst	20

A Obsah přiloženého média

```
mono-OdbornaPrace/  
├─ prace/                                # Rukopis a sazba odborné práce  
│   └─ kapitoly/                        # Texty jednotlivých kapitol (01-uvod  
až 06-prilohy)  
│   └─ lib/odborna-prace.typ            # Sazební šablona, recenzní značky a  
formátování GJKT  
│   └─ img/                            # Vektorová procesní schémata (SVG) a  
grafické podklady  
│   └─ fonts/                          # Metricky shodná patková písma  
(Caladea)  
│   └─ bib/references.bib               # Bibliografická databáze citovaných  
zdrojů  
│   └─ scripts/                        # Pomocné a náhledové skripty  
│   └─ main.typ                        # Hlavní řídicí dokument sazby  
│   └─ metadata.typ                    # Údaje o autorovi, vedoucím a  
anotace  
│   └─ Makefile                        # Příkazy pro automatizovanou  
kompilaci a kontrolu  
├─ darkfactory/                         # Zdrojový kód systému DarkFactory  
(submodul)  
│   └─ .github/workflows/              # Sdílené šablony kontinuální  
integrace  
│   └─ src/                            # Výkonné orchestrační jádro systému  
├─ out/prace.pdf                       # Výsledný vysázený tiskový dokument  
v PDF  
└─ README.md                           # Dokumentace a návod na reprodukci  
prostředí
```

Výpis 1: Stromová adresářová struktura odevzdaného elektronického archivu a doprovodných repozitářů.

B Schéma konfiguračního manifestu darkfactory.json

C Sdílené workflow pro GitHub Actions

D Systémové prompty plánovacího a kódovacího agenta

E Protokol revizních značek v sazebním systému Typst

```
#import "lib/odborna-prace.typ": note, issue, alert, critique, added,
draft, confirmed, diff

// --- 1. Panely na okraji textu (Callouty) ---
#note[Doplňte porovnání rychlosti kompilace mezi verzemi 0.1 a 0.2.]
#issue[Chybná signatura funkce: chybí povinný parametr timeout.]
#alert[Sekce postrádá shrnutí naměřených výsledků před diskusí.]
#critique[Metodologická absence baseline prokazující přínos nového
modulu.]
#blue-note[Metodické vymezení: rozsah práce a oddělení agentního
inženýrství od strojového učení.]

// --- 2. Textové revizní funkce (zvýraznění v toku textu) ---
#draft[Tento odstavec tvoří neověřený koncept čekající na schválení.]
#added[Nově vygenerovaná sekce automaticky začleněná agentem.]
#confirmed[Uživatelem zkontrolovaný text, který ještě nebyl finalizován.]
#removed[Zastaralý text navržený k odstranění.]
#diff[Původní chybné znění textu.][Nové opravené znění textu po revizi.]
```

Výpis 2: Ukázka zápisu a použití revizních značek a textových funkcí v jazyce Typst.

Ukázky textových zvýrazňovacích a srovnávacích funkcí v toku odstavce:

- **Neověřený koncept** (`#draft` / `#unconfirmed`):
- **Nově přidáný text** (`#added`): Tato pasáž byla nově vygenerována autonomním agentem na základě požadavku.
- **Potvrzený text** (`#confirmed`): Text byl předběžně odsouhlasen uživatelem, čeká na finální začištění.
- **Navrženo k odstranění** (`#removed`):
- **Srovnávací diff** (`#diff`): Nové přesné, fakticky a formálně ověřené znění pasáže.
- **Čistý neoznačený text**: Představuje finální, autorsky schválený text v hlase autora bez jakéhokoli podbarvení.

Seznam zdrojů

1. CHACON, Scott a STRAUB, Ben. *Pro Git*. 2. New York : Apress, 2014. IS-BN 978-1-4842-0076-6.
2. VASWANI, Ashish, SHAZEER, Noam, PARMAR, Niki, USZKOREIT, Jakob, JONES, Llion, GOMEZ, Aidan N., KAISER, Łukasz a POLOSUKHIN, Illia. Attention Is All You Need. *arXiv preprint arXiv:1706.03762*. Online. 2017. Available from: <https://arxiv.org/abs/1706.03762>
3. MARIUS, Patrik. DarkFactory: autonomous, governed software engineering pipelines. Online. 2026. [Accessed 8 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory>
4. HUMBLE, Jez a FARLEY, David. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston : Addison-Wesley, 2010. ISBN 978-0-321-60191-9.
5. Deepseek Harness. *arXiv preprint arXiv:2608.25512*. Online. 2026. Available from: <https://arxiv.org/abs/2608.25512>
6. ANTHROPIC. Model Context Protocol documentation. Online. 2026. Available from: <https://modelcontextprotocol.io/docs/2026-07-28/getting-started/intro#explore-mcp>
7. ANTHROPIC. Prompt Engineering overview. Online. 2026. Available from: <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview>
8. YAO, Shunyu, ZHAO, Jeffrey, YU, Dian, DU, Nan, SHAFRAN, Izhak, NARASIMHAN, Karthik a CAO, Yuan. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*. Online. 2022. Available from: <https://arxiv.org/abs/2210.03629>
9. LIU, Nelson F., LIN, Kevin, HEWITT, John, PARANJAPE, Ashwin, BEVILACQUA, Michele, PETRONI, Fabio a LIANG, Percy. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*. Online. 2024. Vol. 12, p. 157–173. Available from: <https://arxiv.org/abs/2307.03172>
10. DAO, Tri, FU, Daniel Y., ERMON, Stefano, RUDRA, Atri a RÉ, Christopher. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *Advances in Neural Information Processing Systems*. Online. 2022. Vol. 35, p. 16344–16359. Available from: <https://arxiv.org/abs/2205.14135>
11. AINSLIE, Joshua, LEE-THORP, James, JONG, Michiel de, ZEMLYANSKIY, Yury, LEBRÓN, Federico a SANGHAI, Sumit. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv preprint arXiv:2305.13245*. Online. 2023. Available from: <https://arxiv.org/abs/2305.13245>